

Modül 11

Resource API

Derinlemesine Pattern'ler

Modül: 11 / 30

Ders Sayısı: 6 ders

Seviye: İleri

Versiyon: Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

📦 Modül 11'e Hoş Geldin

Modül 5 ve 10'da `resource()` ve `httpResource()`'u gördük. Bu modül onların production-grade pattern'lerini derinlemesine işliyor: pagination, search, mutations, error recovery, skeleton loading.

Bu modülün sonunda yapabileceklerin:

- ✓ `resource()` vs `httpResource()` farkını net bilmek
- ✓ Pagination pattern uygulamak
- ✓ Debounced search + filter implement etmek
- ✓ Mutation'lar sonrası state sync yapmak
- ✓ Error recovery ve retry strategies
- ✓ Loading skeleton UI yazmak

resource() vs httpResource()

İkisi de async data için. Hangisi ne zaman?

❏ Karşılaştırma

Konu	resource()	httpResource()
Tür	Generic async	HTTP-specific
Loader	Custom (any async)	HttpClient
Cancellation	AbortSignal	Auto
Use case	WebSocket, IndexedDB, vs	REST API

❏ resource() — Generic

```
import { resource, signal } from '@angular/core';

userId = signal('123');

userResource = resource({
  request: () => ({ id: this.userId() }),
  loader: async ({ request, abortSignal }) => {
    const res = await fetch(`/api/users/${request.id}`, {
      signal: abortSignal
    });
    if (!res.ok) throw new Error('Failed');
    return res.json();
  }
});

// IndexedDB örneği
dbResource = resource({
  request: () => ({ key: this.key() }),
  loader: async ({ request }) => {
    const db = await openDB('myDB');
    return db.get('users', request.key);
  }
});
```

❏ httpResource() — HTTP

```
import { httpResource } from '@angular/common/http';

userId = signal('123');

// Basit
user = httpResource<User>(() => `/api/users/${this.userId()}`);

// Detaylı config
user = httpResource<User>(() => ({
  url: `/api/users/${this.userId()}`,
  method: 'GET',
  params: { include: 'profile' },
  headers: { 'X-Custom': 'value' },
  context: new HttpContext().set(SKIP_AUTH, false),
}));
```

Pagination Pattern

Sayfalı veri yönetimi.

□ Temel Pagination

```
@Component({...})
export class UsersListComponent {
  page = signal(1);
  pageSize = signal(20);

  users = httpResource<PagedResponse<User>>(() => ({
    url: '/api/users',
    params: {
      page: this.page(),
      size: this.pageSize(),
    }
  }));

  nextPage() { this.page.update(p => p + 1); }
  prevPage() { this.page.update(p => Math.max(1, p - 1)); }
  goToPage(p: number) { this.page.set(p); }
}

interface PagedResponse<T> {
  items: T[];
  totalCount: number;
  page: number;
  pageSize: number;
}
```

□ Template

```

@if (users.isLoading()) {
  <app-spinner />
} @else if (users.value(); as data) {
  @for (user of data.items; track user.id) {
    <app-user-card [user]="user" />
  }

  <div class="pagination">
    <button (click)="prevPage()" [disabled]="page() === 1"></button>
    <span>Sayfa {{ page() }} / {{ Math.ceil(data.totalCount / pageSize()) }}</span>
    <button (click)="nextPage()" [disabled]="page() * pageSize() >=
data.totalCount"></button>
  </div>
}

```

∞ Infinite Scroll Pattern

```

@Component({...})
export class InfiniteListComponent {
  page = signal(1);
  allItems = signal<Item[]>([]);

  pageResource = httpResource<Item[]>(() => `/api/items?page=${this.page()}`);

  constructor() {
    // Yeni sayfa geldiğinde mevcut listeye ekle
    effect(() => {
      const newItems = this.pageResource.value();
      if (newItems) {
        this.allItems.update(items => [...items, ...newItems]);
      }
    });
  }

  loadMore() {
    if (!this.pageResource.isLoading()) {
      this.page.update(p => p + 1);
    }
  }
}

```

Search & Filter

Debounce + distinct + auto-cancellation.

▣ Debounced Search

```
@Component({...})
export class SearchComponent {
  searchTerm = signal('');
  debouncedTerm = signal('');

  constructor() {
    // Debounce: 300ms yazma duraklarsa update
    effect((onCleanup) => {
      const term = this.searchTerm();
      const id = setTimeout(() => {
        this.debouncedTerm.set(term);
      }, 300);
      onCleanup(() => clearTimeout(id));
    });
  }

  results = httpResource<SearchResult[]>(() => {
    const q = this.debouncedTerm();
    if (q.length < 2) return undefined; // Skip
    return `/api/search?q=${encodeURIComponent(q)}`;
  });

  onInput(e: Event) {
    this.searchTerm.set((e.target as HTMLInputElement).value);
  }
}
```

▣ Auto Cancellation

httpResource otomatik cancellation yapar. Yeni request geldiğinde eski request iptal edilir. Race condition yok.

```
// Sen yazıyorsun: "a" → "an" → "ang" → "angular"
// Her seferinde yeni request
// Eski request'ler OTOMATİK iptal edilir
// Son request'in cevabı gelir, eskilerin değil
```

▣ Multi-Filter

```
@Component({...})
export class ProductListComponent {
  searchTerm = signal('');
  category = signal<string | null>(null);
  minPrice = signal<number | null>(null);
  maxPrice = signal<number | null>(null);
  sortBy = signal<'name' | 'price'>('name');

  products = httpResource<Product[]>(() => ({
    url: '/api/products',
    params: {
      ...(this.searchTerm() && { q: this.searchTerm() }),
      ...(this.category() && { category: this.category() }),
      ...(this.minPrice() && { minPrice: this.minPrice() }),
      ...(this.maxPrice() && { maxPrice: this.maxPrice() }),
      sort: this.sortBy(),
    }
  }));
}
```


Mutations & State Sync

Veri değiştirdikten sonra cache'i nasıl güncelleyeceğiz?

❑ reload() — Manuel Refresh

```
@Component({...})
export class UsersComponent {
  private http = inject(HttpClient);

  users = httpResource<User[]>('/api/users');

  async deleteUser(id: string) {
    await firstValueFrom(this.http.delete(`/api/users/${id}`));
    this.users.reload(); // Listeyi yenile
  }

  async createUser(data: CreateUserDto) {
    await firstValueFrom(this.http.post('/api/users', data));
    this.users.reload();
  }
}
```

❑ Optimistic Update Pattern

```

@Injectables({ providedIn: 'root' })
export class TodoService {
  private http = inject(HttpClient);

  todos = httpResource<Todo[]>('/api/todos');

  async toggleTodo(id: string) {
    // 1. Snapshot al (rollback için)
    const original = this.todos.value();
    if (!original) return;

    // 2. Optimistic update (henüz manuel API yok, manuel state)
    // Not: httpResource'un value'sini direkt set edemezsin
    // Local state ile orchestrate et:

    const todoToToggle = original.find(t => t.id === id);
    if (!todoToToggle) return;

    try {
      await firstValueFrom(
        this.http.patch(`/api/todos/${id}`, {
          completed: !todoToToggle.completed
        })
      );

      // 3. Sunucudan tekrar yükle (consistent state)
      this.todos.reload();
    } catch (e) {
      alert('Güncelleme başarısız');
    }
  }
}

```

❑ Mutation Service Pattern

```
// State + mutations bir arada
@Injectables({ providedIn: 'root' })
export class TodoStore {
  private http = inject(HttpClient);

  // Read state
  todos = httpResource<Todo[]>('/api/todos');

  // Computed
  activeTodos = computed(() =>
    this.todos.value()?.filter(t => !t.completed) ?? []
  );

  // Mutations
  async addTodo(title: string) {
    await firstValueFrom(this.http.post('/api/todos', { title }));
    this.todos.reload();
  }

  async deleteTodo(id: string) {
    await firstValueFrom(this.http.delete(`/api/todos/${id}`));
    this.todos.reload();
  }
}
```

Error Recovery & Retry

Hatalar olduğunda kullanıcıya iyi UX vermek.

❑ Manual Retry

```
@if (users.error()) {  
  <div class="error-state">  
    <p>Yüklenemedi: {{ users.error() }}</p>  
    <button (click)="users.reload()">Tekrar Dene</button>  
  </div>  
}
```

❑ Auto Retry with Backoff

```
// Interceptor seviyesinde  
export const retryInterceptor: HttpInterceptorFn = (req, next) => {  
  return next(req).pipe(  
    retry({  
      count: 3,  
      delay: (error, retryCount) => {  
        if (error.status >= 500) {  
          // Exponential backoff: 1s, 2s, 4s  
          return timer(1000 * Math.pow(2, retryCount));  
        }  
        throw error;  
      }  
    })  
  );  
};
```

❑ Stale-While-Revalidate Pattern

Önce eski veriyi göster, arka planda yenisini al. Cache invalidation strategy.

```
@Injectable({ providedIn: 'root' })
export class CachedDataService {
  private cache = signal<Data | null>(null);
  private loading = signal(false);

  // Initial load
  initialLoad = httpResource<Data>('/api/data');

  constructor() {
    effect(() => {
      const fresh = this.initialLoad.value();
      if (fresh) this.cache.set(fresh);
    });
  }

  // Force refresh
  async refresh() {
    this.loading.set(true);
    try {
      const fresh = await firstValueFrom(this.http.get<Data>('/api/data'));
      this.cache.set(fresh);
    } finally {
      this.loading.set(false);
    }
  }
}
```

Loading Skeleton & UX

Spinner yerine skeleton ile daha iyi perceived performance.

□ Skeleton Component

```
@Component({
  selector: 'app-skeleton',
  template: `
    <div class="skeleton" [style.width.px]="width()" [style.height.px]="height()">
      </div>
  `,
  styles: [`
    .skeleton {
      background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%);
      background-size: 200% 100%;
      animation: shimmer 1.5s infinite;
      border-radius: 4px;
    }
    @keyframes shimmer {
      0% { background-position: 200% 0; }
      100% { background-position: -200% 0; }
    }
  `]
})
export class SkeletonComponent {
  width = input(100);
  height = input(20);
}
```

□ List Skeleton

```
<!-- users.html -->
@if (users.isLoading()) {
  <!-- 5 fake item göster -->
  @for (i of [1,2,3,4,5]; track i) {
    <div class="user-card-skeleton">
      <app-skeleton [width]="60" [height]="60" />
      <div>
        <app-skeleton [width]="200" [height]="16" />
        <app-skeleton [width]="150" [height]="12" />
      </div>
    </div>
  }
} @else if (users.value(); as data) {
  @for (user of data; track user.id) {
    <app-user-card [user]="user" />
  }
}
```

□ Stagger Animation

```
.user-card-skeleton:nth-child(1) { animation-delay: 0s; }
.user-card-skeleton:nth-child(2) { animation-delay: 0.1s; }
.user-card-skeleton:nth-child(3) { animation-delay: 0.2s; }
.user-card-skeleton:nth-child(4) { animation-delay: 0.3s; }
.user-card-skeleton:nth-child(5) { animation-delay: 0.4s; }
```

□ UX Best Practices

- ✓ Skeleton > Spinner — kullanıcı içeriğin nerede geleceğini görür
- ✓ Skeleton içerik boyutuyla aynı olmalı — layout shift olmasın
- ✓ Animasyon hafif olsun — distract etmemeli
- ✓ 200ms'den kısa loading için skeleton göstermeye gerek yok

Modül 11 Özeti

Resource API'sını production seviyesinde kullanabilirsin artık. Pagination, search, mutations, error recovery, skeleton UI — modern web app'in tüm gereksinimleri.

Neler Öğrendin?

- ✓ resource() vs httpResource() farkı
- ✓ Pagination ve infinite scroll pattern'leri
- ✓ Debounced search + multi-filter
- ✓ Mutations sonrası state sync (reload, optimistic)
- ✓ Error recovery ve auto retry
- ✓ Loading skeleton UI ile iyi UX

Sıradaki Modül

Modül 12 — Pipes & Custom Pipes. Veri dönüşümleri için Angular'ın güçlü pipe sistemi. Date, currency, async, custom pipes.